

SOICT



MOTIF: Multi-strategy Optimization via Turn-based Interactive Framework

Nguyen Viet Tuan Kiet¹, Dao Van Tung¹, Tran Cong Dao², Huynh Thi Thanh Binh¹

¹Hanoi University of Science and Technology

²FPT Software AI Center

Singapore EXPO • Jan 23, 2026

Contents

1. Introduction
2. Multi-strategy Optimization
3. Turn-based Interactive Framework
4. Experimental Results
5. Discussions

1. Introduction

1.1. Combinatorial Optimization

1.2. Automatic Heuristic Design

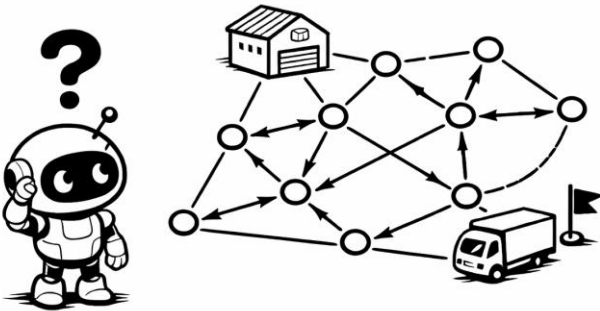
1.3. Limitations

1.4. Research Questions

Combinatorial Optimization

Routing

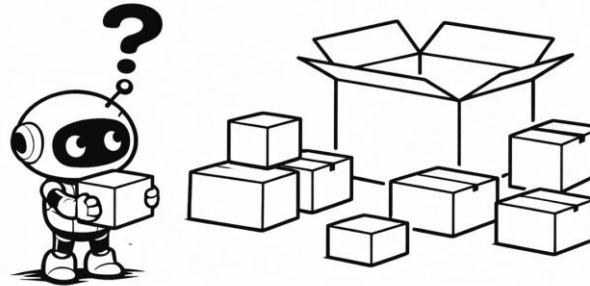
TSP, VRP, OP



Logistics, delivery routing, ride-sharing, network design

Packing

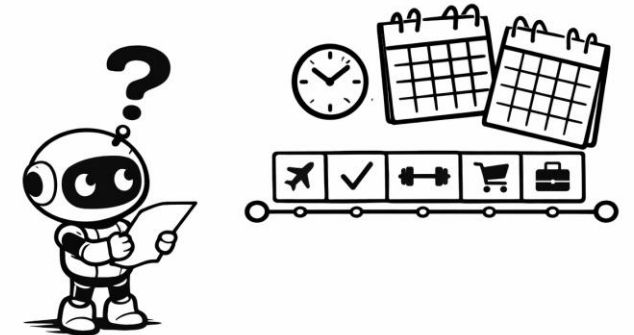
Online / Offline BPP



Warehousing, container loading, cloud resource allocation

Scheduling

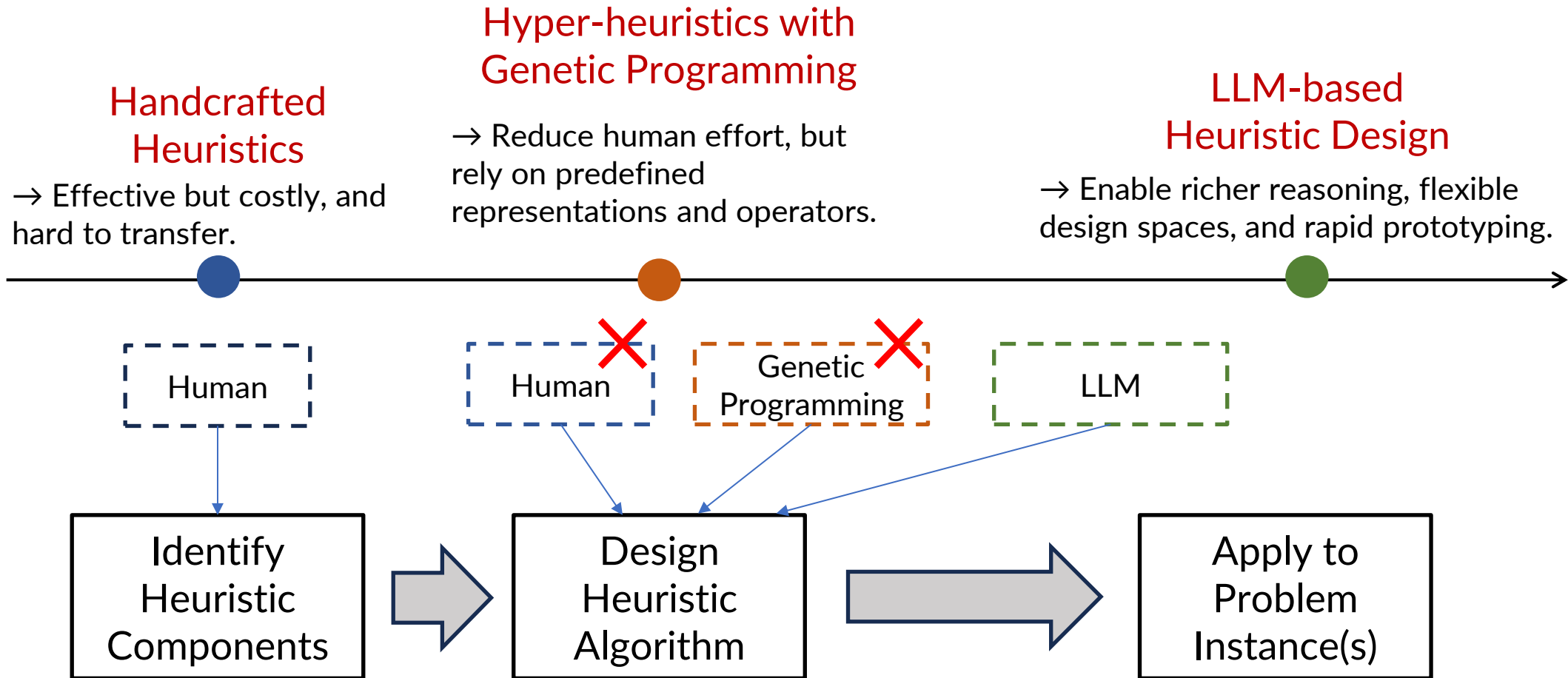
JSSP, FSSP



Manufacturing lines, job allocation, timetabling, cloud scheduling

→ NP-hard problems, solved in practice by *heuristic* and *metaheuristic* solvers.

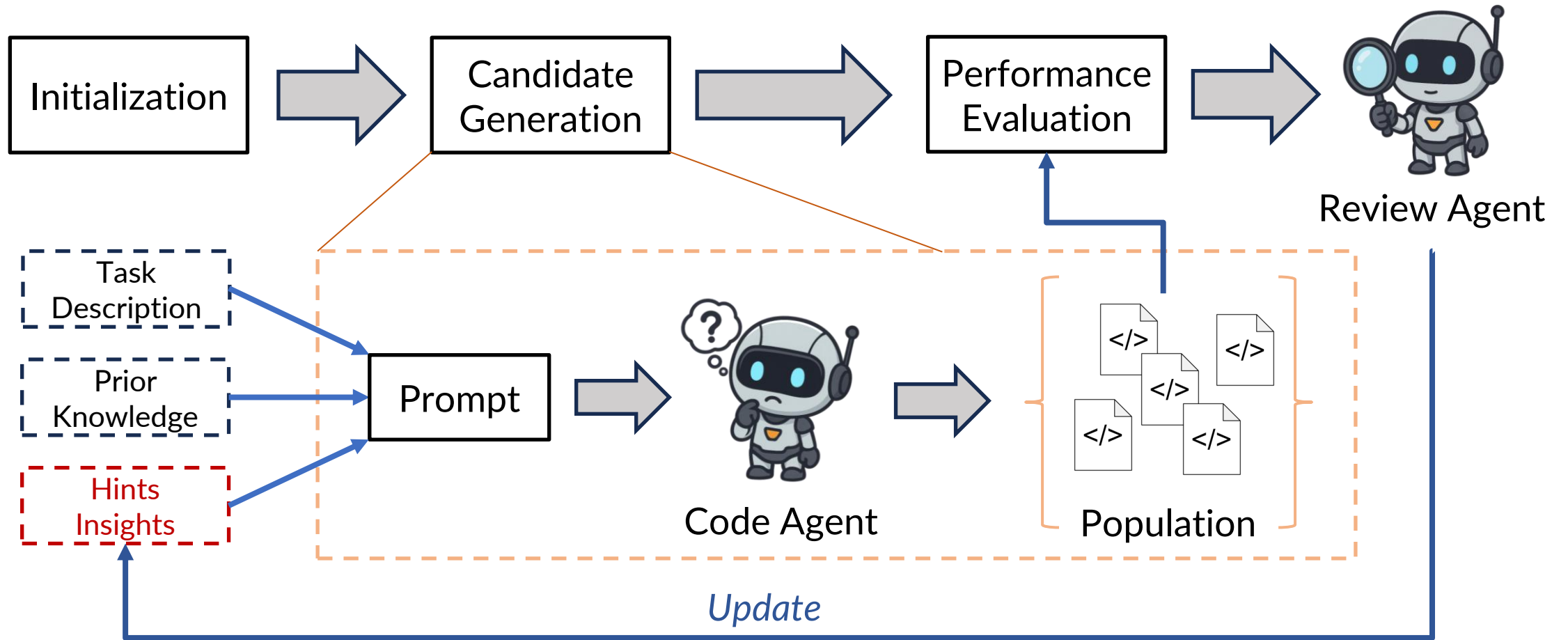
Automatic Heuristic Design



(1) Burke et al. (2009), "Exploring Hyper-heuristic Methodologies with Genetic Programming", Computational Intelligence.

(2) Liu et al. (2023), "Algorithm Evolution Using Large Language Model", Preprint

Current Trends: LLM + EA

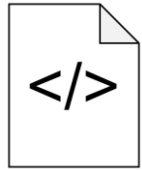


(3) Liu et al. (2023), "Algorithm Evolution Using Large Language Model", Preprint

(4) Liu et al. (2024), "Evolution of Heuristics: Towards Efficient Automatic Algorithm Design Using Large Language Model", ICML 2024

(5) Ye et al. (2024), "ReEvo: Large Language Models as Hyper-Heuristics with Reflective Evolution, NeurIPS 2024

Limitations

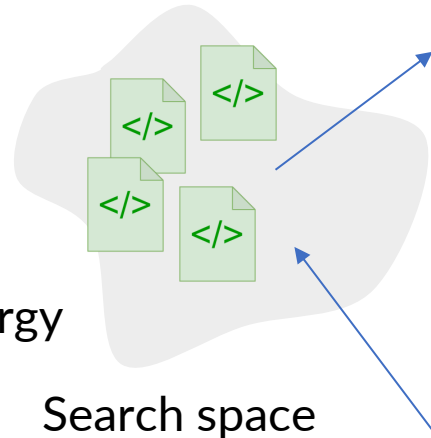


► Focus on single heuristic component

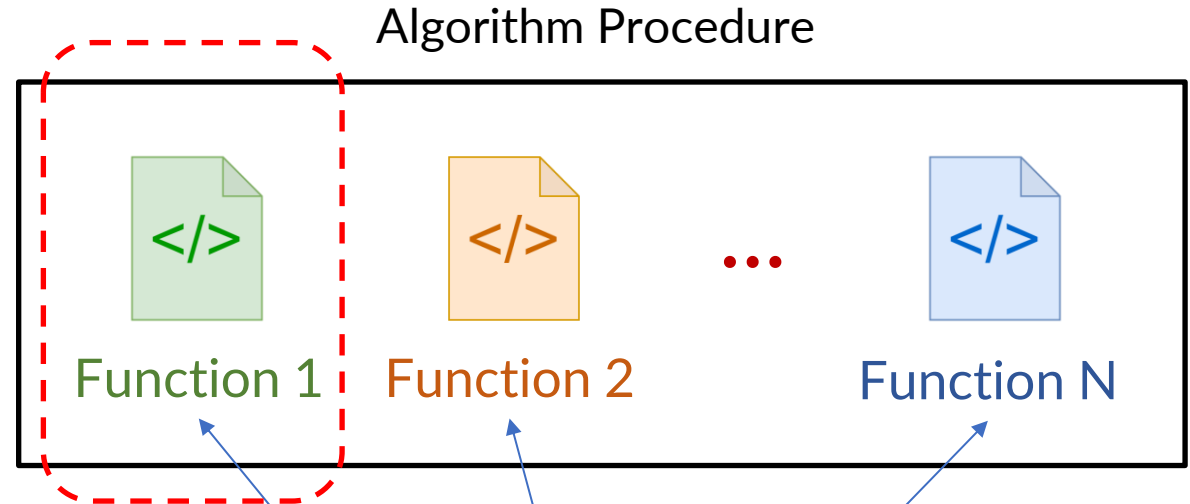
- Only **one function** is optimized
- Other solver components remain fixed
- Interactions between functions are ignored

→ Consequences

- Narrow search space
- Limited solver creativity
- Missed component synergy



Search space



Algorithm Procedure

Function 1

Function 2

Function N

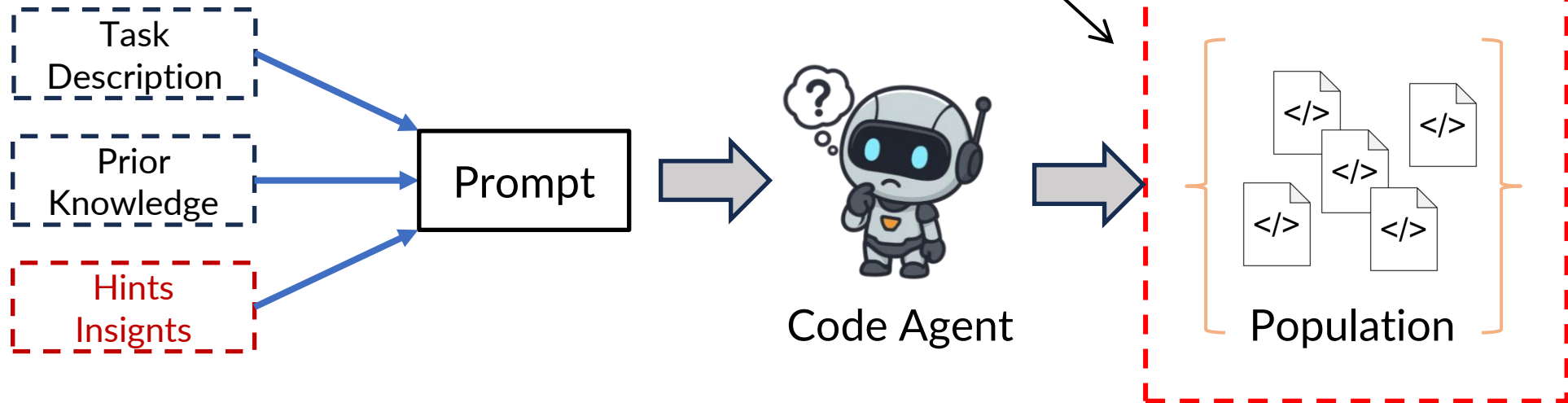
Solver = multiple heuristic components
e.g. construction, scoring, update, repair...

Select an
heuristic

Limitations

► Population-based search prunes promising ideas early

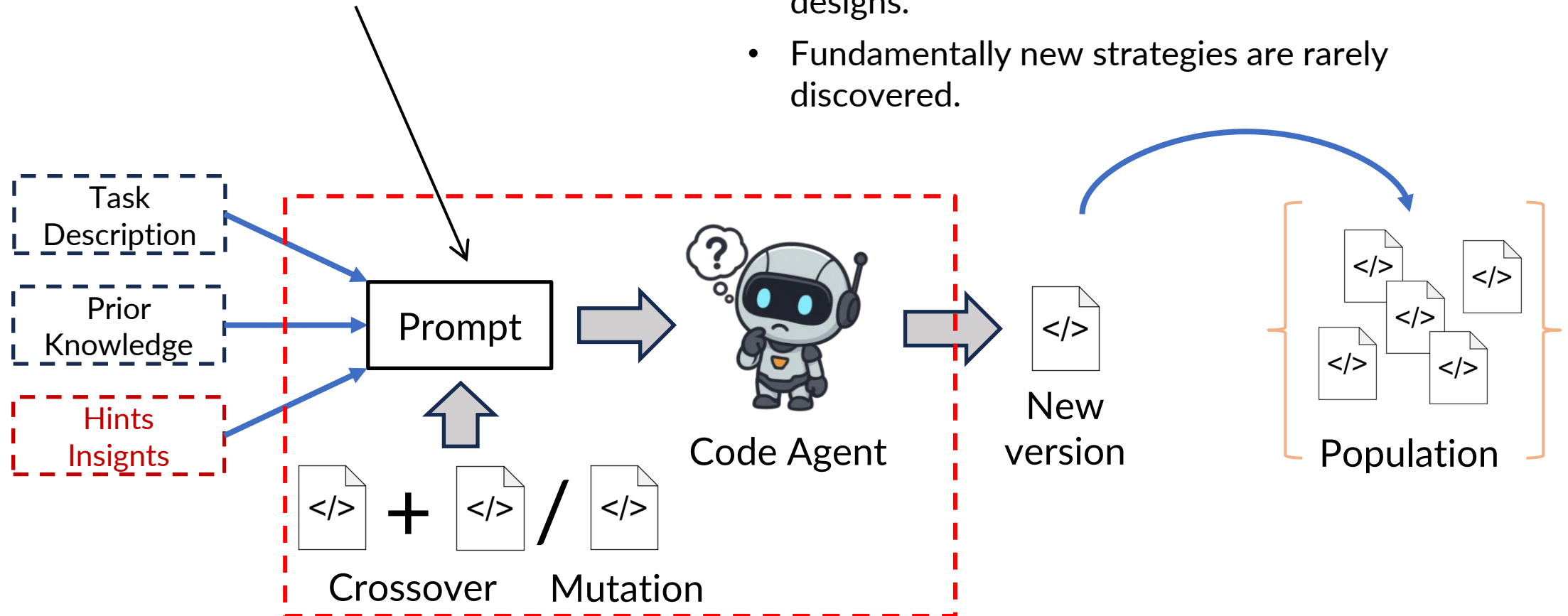
- Promising heuristics are discarded in early generations.
- Search is constrained by predefined operators and representations.



Limitations

► Search is constrained by crossover and mutation operators

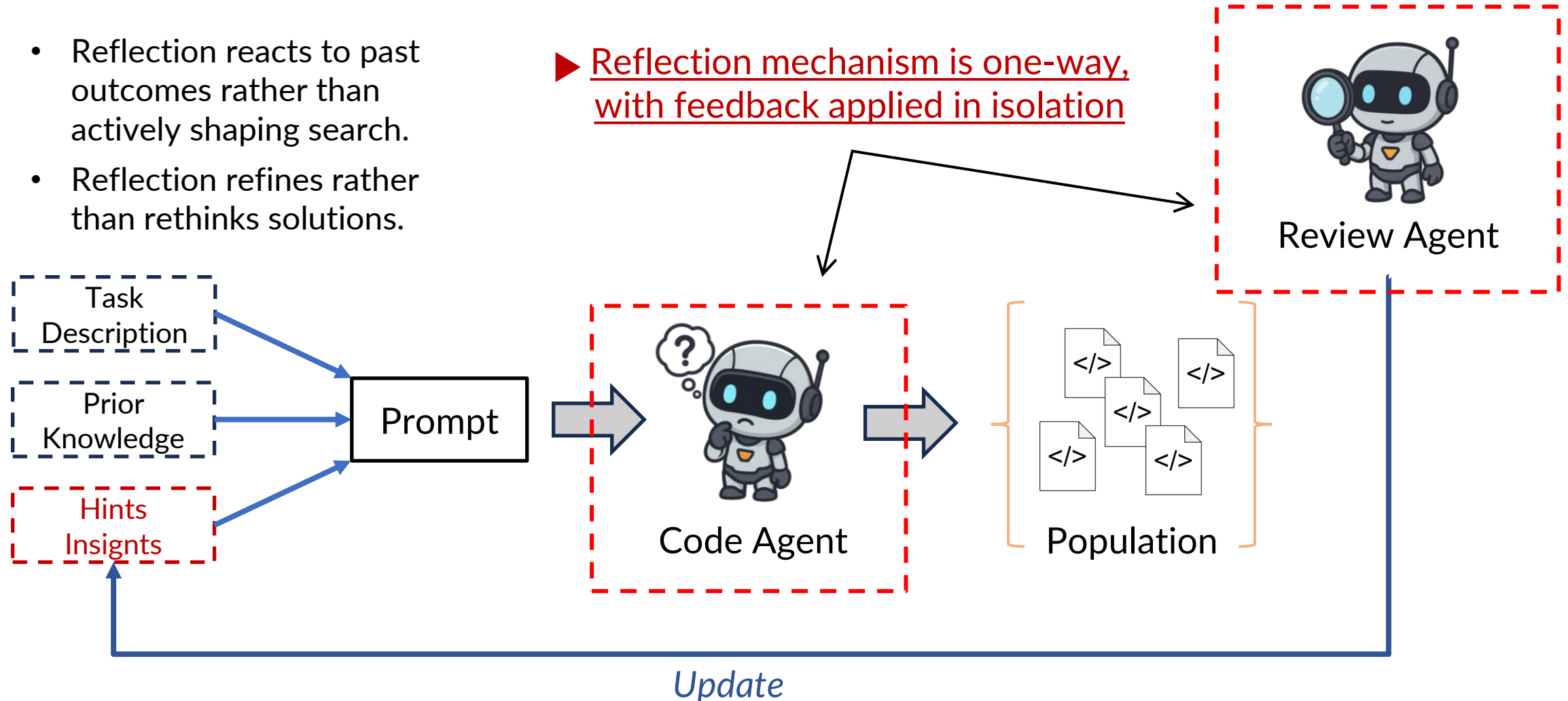
- New heuristics are composed from existing ones.
- Changes are incremental and confined to nearby designs.
- Fundamentally new strategies are rarely discovered.



Limitations

- Reflection reacts to past outcomes rather than actively shaping search.
- Reflection refines rather than rethinks solutions.

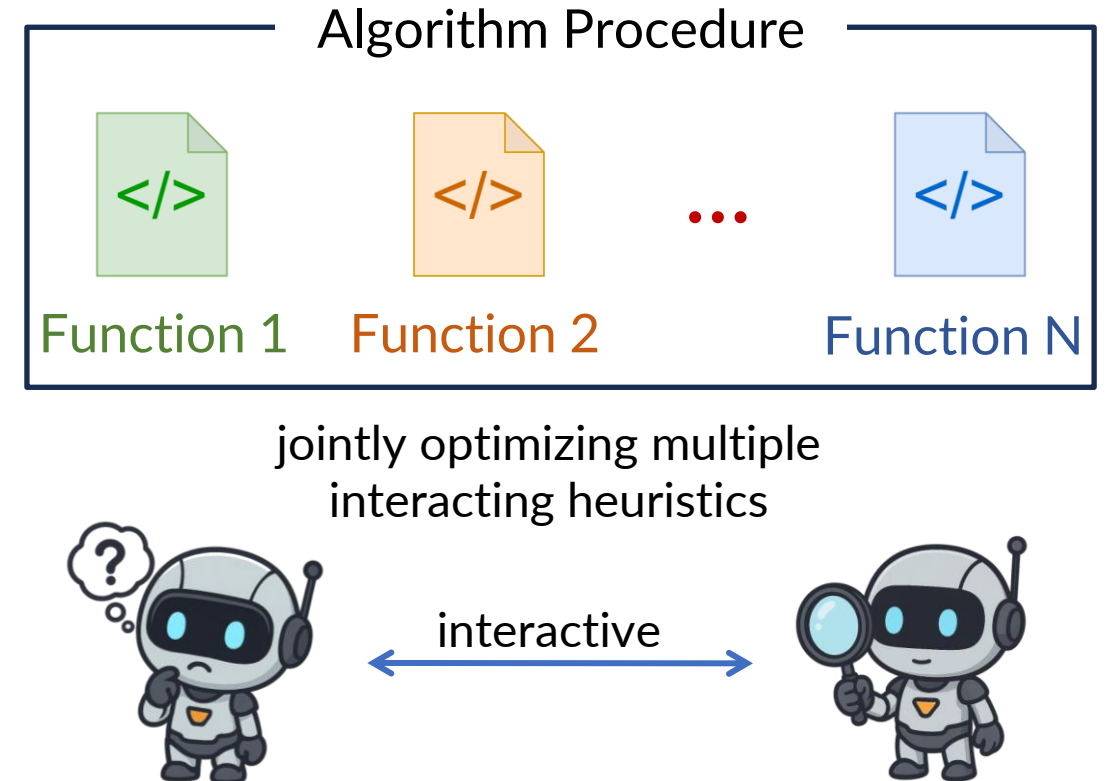
► Reflection mechanism is one-way, with feedback applied in isolation



Research Questions

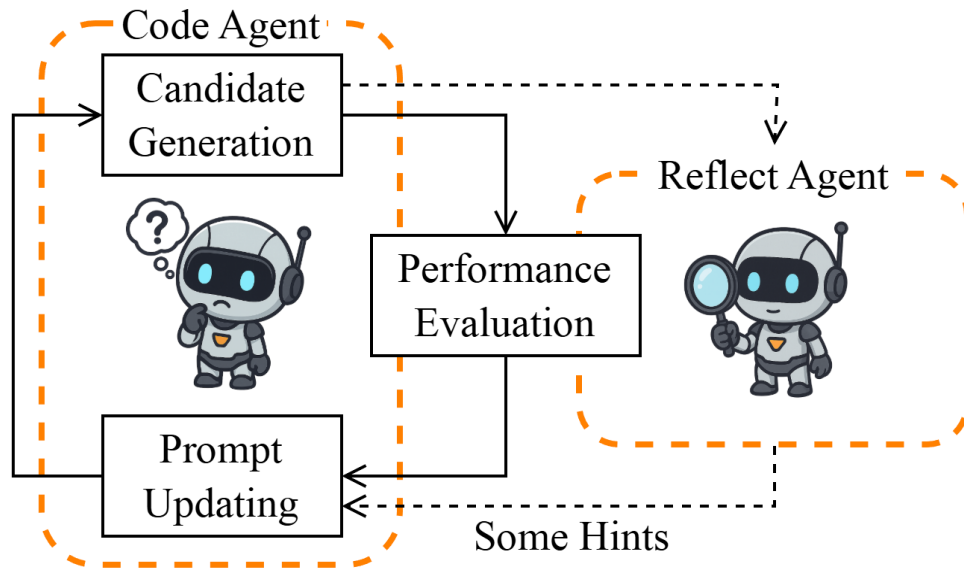
1. How to jointly optimize multiple interacting heuristics?
2. How to introduce structured interaction into AHD?
3. How to overcome one-way reflection and operator bias?
4. How to better exploit LLM reasoning for solver design?

→ These questions motivate a new paradigm for automatic heuristic design.

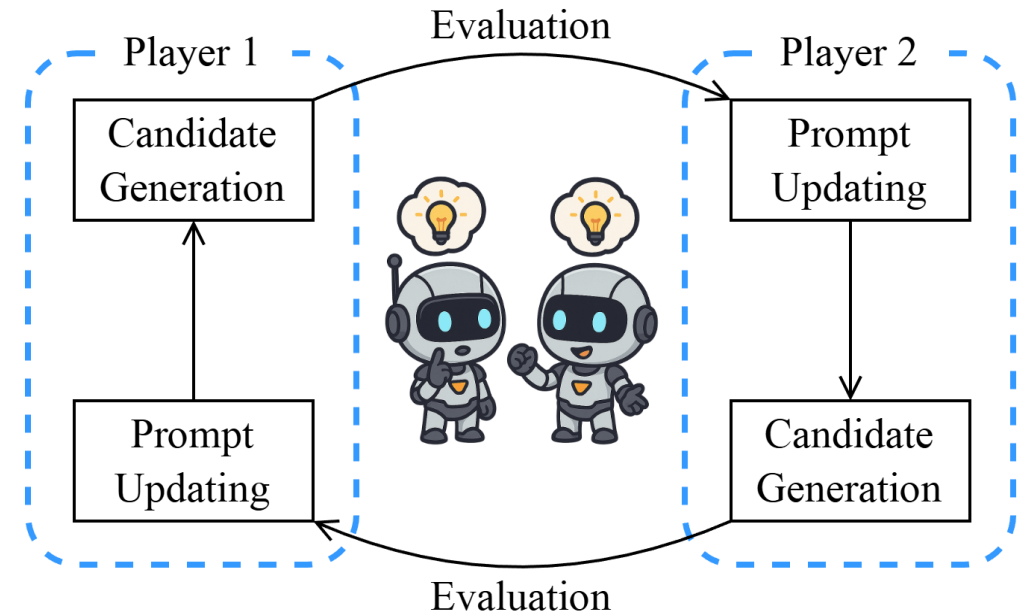


Prior Work vs. Our Perspective

(a) Monolithic reflective pipeline^(3, 4, 5)



(b) Turn-based interactive framework (ours)⁽⁶⁾



→ NEW problem, NEW mechanism, NEW operators

(3) Liu et al. (2023), "Algorithm Evolution Using Large Language Model"

(4) Liu et al. (2024), "Evolution of Heuristics: Towards Efficient Automatic Algorithm Design Using Large Language Model", ICML 2024

(5) Ye et al. (2024), "ReEvo: Large Language Models as Hyper-Heuristics with Reflective Evolution, NeurIPS 2024

(6) Nguyen et al. (2026), "MOTIF: Multi-strategy Optimization via Turn-based Interactive Framework", AAAI 2026

2. Multi-strategy Optimization

2.1. Problem Domain

2.2. Solvers

2.3. Strategy Spaces

2.4. Objective

Domain: Instance & Solution

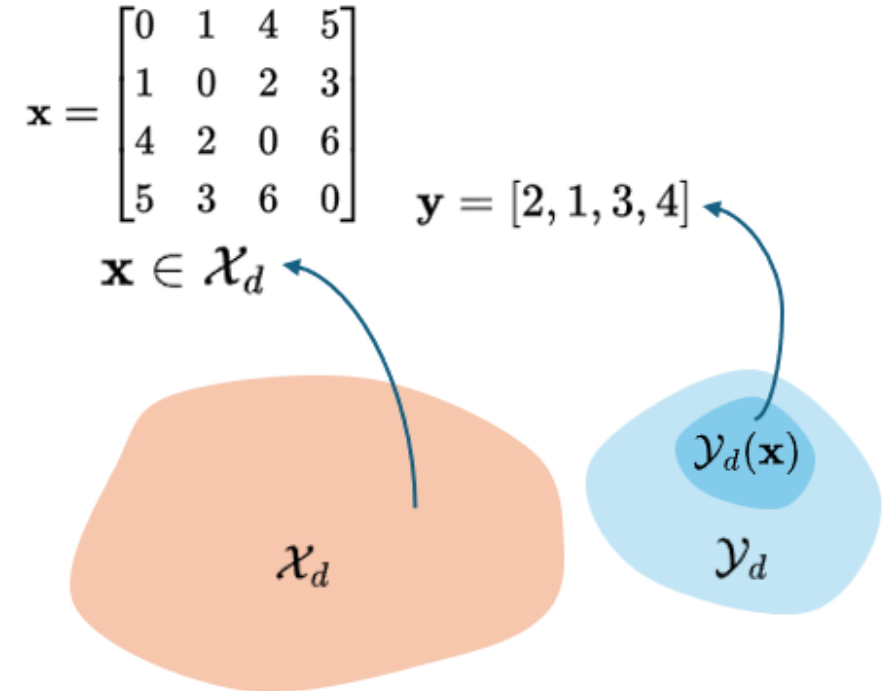
Definition 1.

- Problem domain $d : (\mathcal{X}_d, \mathcal{Y}_d, f_d)$
- Instance space $\mathcal{X}_d$
- An instance $\mathbf{x} \in \mathcal{X}_d$
- Solution space $\mathcal{Y}_d$
- Objective function $f_d : \mathcal{X}_d \times \mathcal{Y}_d \rightarrow \mathbb{R}$

For each instance $\mathbf{x}$, there exists a feasible solution set: $\mathcal{Y}_d(\mathbf{x}) \subset \mathcal{Y}_d$

The objective for a given instance $\mathbf{x}$ is:

$$\mathbf{y}^* = \arg \min_{\mathbf{y} \in \mathcal{Y}_d(\mathbf{x})} f_d(\mathbf{x}, \mathbf{y})$$



$$f_d(\mathbf{x}, \mathbf{y}) = \sum_{i=1}^N \mathbf{x}[\mathbf{y}[i], \mathbf{y}[i+1]], \text{ where } \mathbf{y}[N+1] = \mathbf{y}[1]$$

In the TSP domain, each instance is a distance matrix between cities, and each solution is a permutation of the cities.

Solvers & Strategies

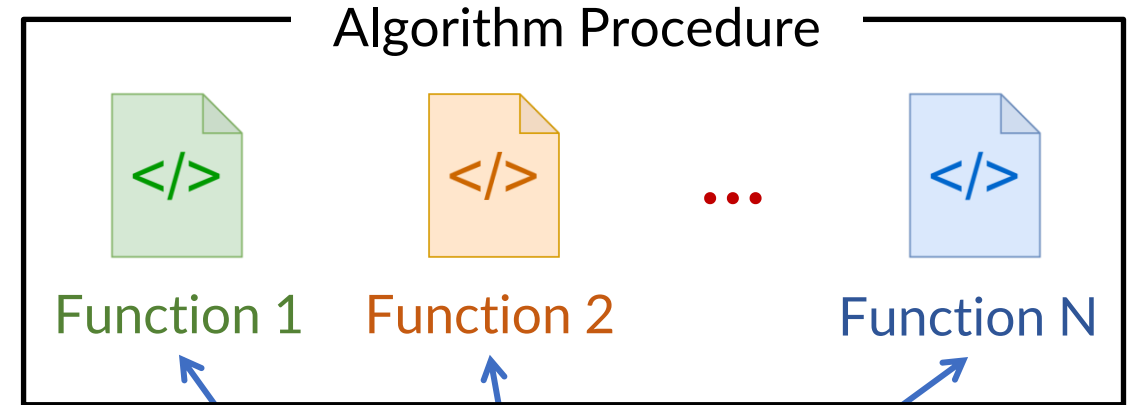
Definition 2.

- A solver s is an algorithmic framework that produces a feasible solution y for each instance x .
- A solver is supported by a set of rules or components (strategies)

$$(\pi_1, \pi_2, \dots, \pi_K)$$

- Notation:

$$\mathbf{y} = s(\mathbf{x} | (\pi_1, \pi_2, \dots, \pi_K)) \approx \mathbf{y}^*$$



A “strategy” can be any functional component of an optimization algorithm.

- It can be a computational function.
- It can be a rule or a selection policy.
- It can be a control function.
- ...

Strategy Space

Definition 3.

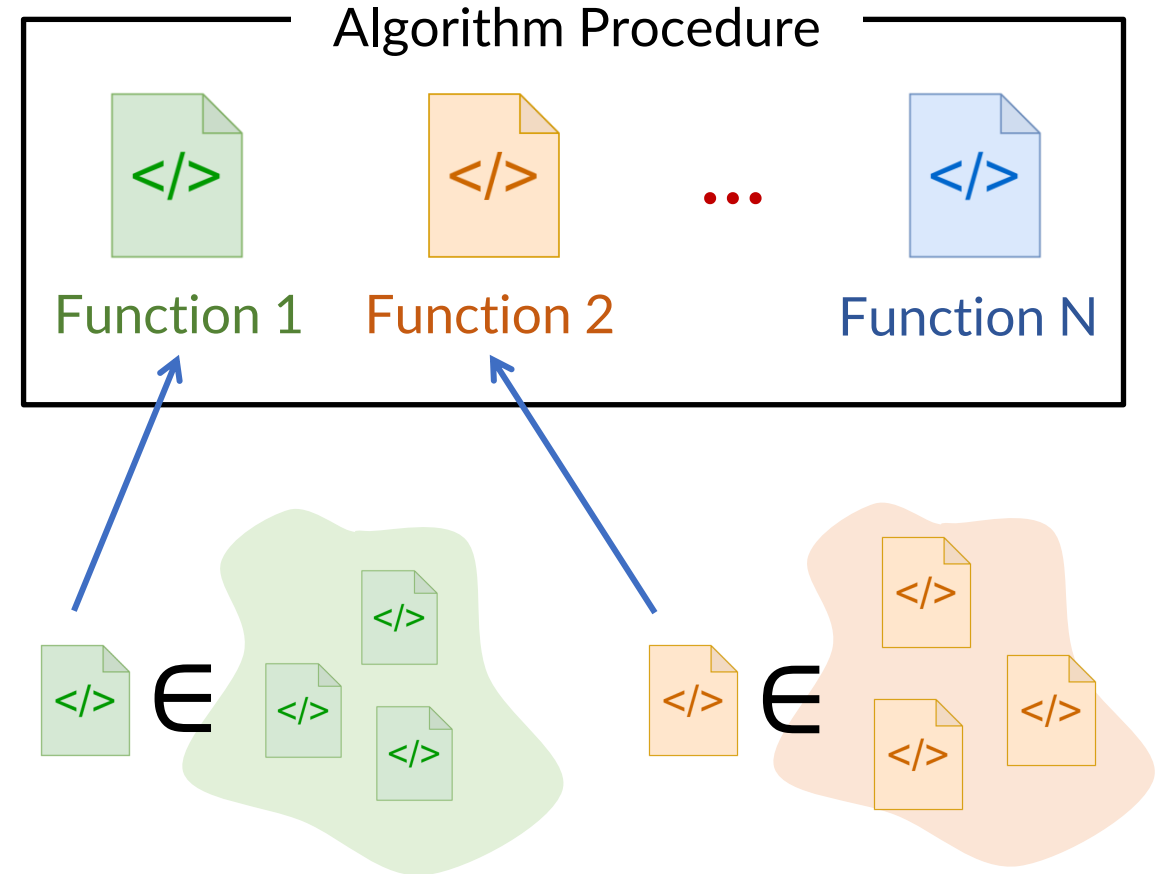
- A strategy $\pi \in \mathcal{S}_\pi$
- A strategy space contains all variants of a given strategy.
- Strategies within the same space may differ in:
 - Internal logic
 - Parameter values
 -
- And are identical in:
 - Functionality
 - Function signature

A solver s is supported by K strategies

$$\mathbf{\Pi} = (\pi_1, \pi_2, \dots, \pi_K)$$

The performance of the solver on instance x is:

$$F_d(\mathbf{x}|\mathbf{\Pi}) = f_d(\mathbf{x}, s(\mathbf{x}|\mathbf{\Pi}))$$



→ Solver performance is fully determined by its strategies.
Optimizing solver performance amounts to optimizing its strategies.

Objective

Definition 4.

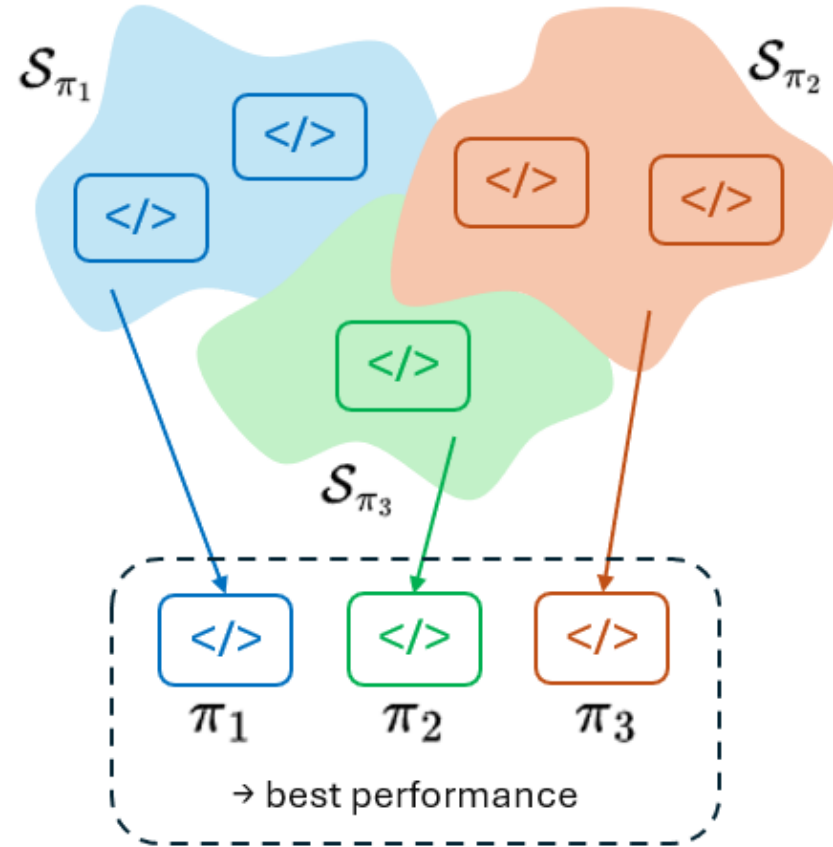
Multi-strategy Optimization Problem.

Objective:

$$\mathbf{\Pi}^* = \arg \min_{\mathbf{\Pi}} \mathbb{E}_{\mathbf{x} \sim \mathcal{X}_d} [F_d(\mathbf{x} | \mathbf{\Pi})]$$

under a finite computational budget.

- This formulation is **more general** than prior approaches.
- It expands the search space, enabling the discovery of novel solver designs that were overlooked by previous studies.



3. Turn-based Interactive Framework

3.1. Architectural Overview

3.2. Component-wise Competition

3.3. System-aware Refinement

3.4. Operators Design

Architectural Overview

ROUND 1

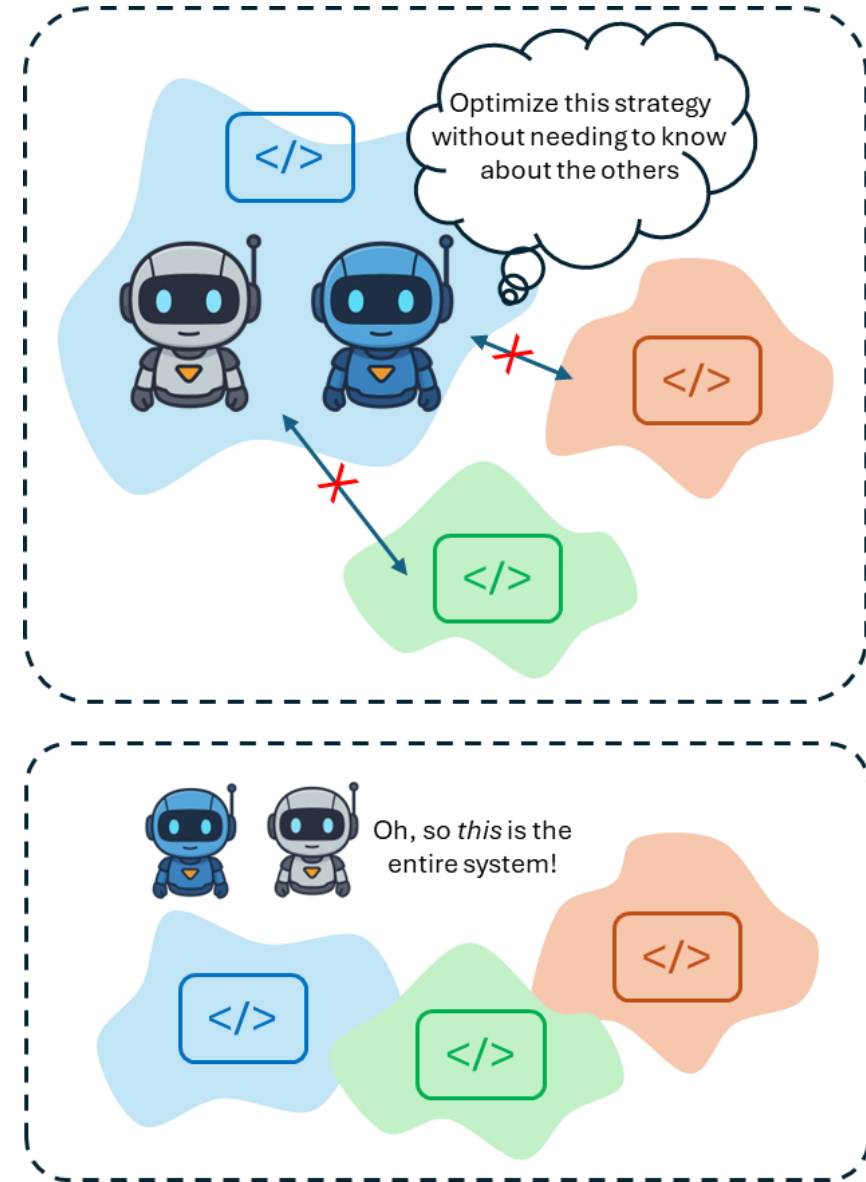
Component-wise Competition

- Independent per-strategy search
- Turn-based competition on one component
- Diverse local improvements.
- No global context needed

ROUND 2

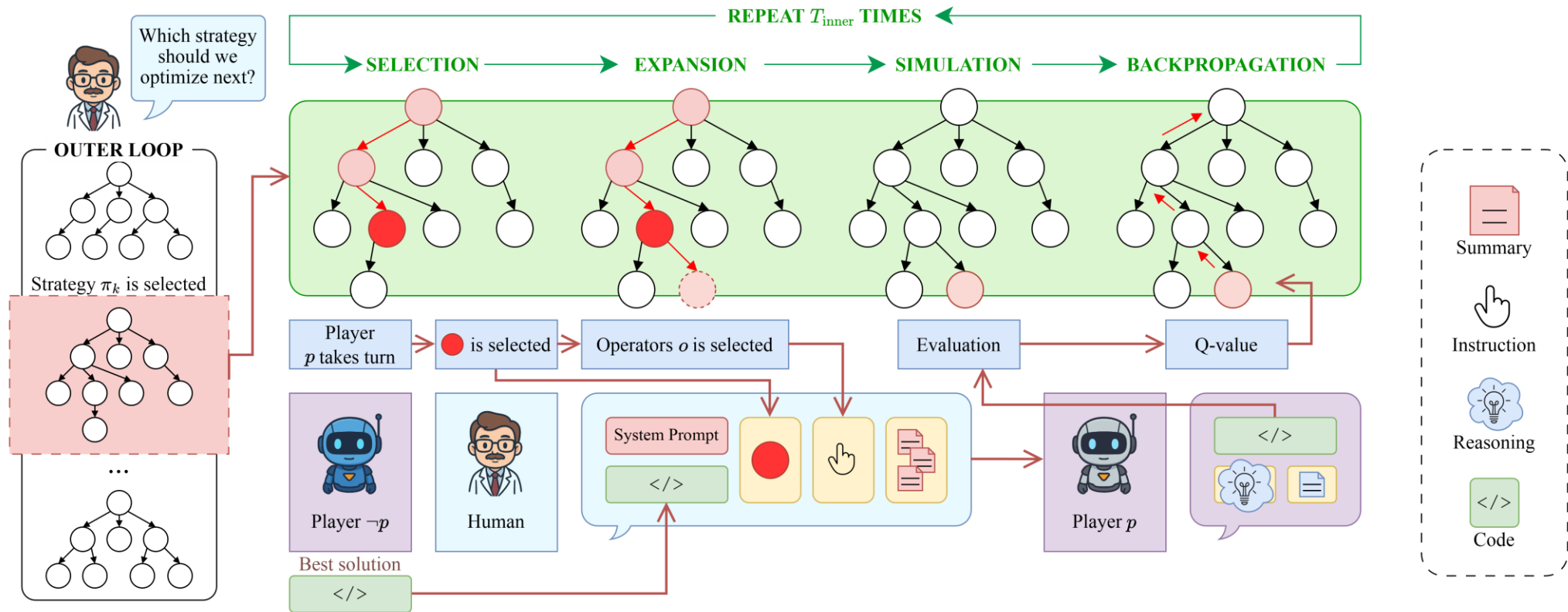
System-aware Refinement

- Full-system visibility
- Contextual refinement for component synergy
- Global improvements beyond local search



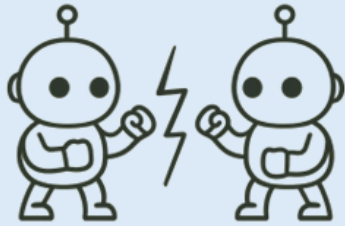
Component-wise Competition

- Per-strategy tree, picked by UCB
- Node = dual-agent state (two implementations)
- Action = operator $\rightarrow$ LLM code revision



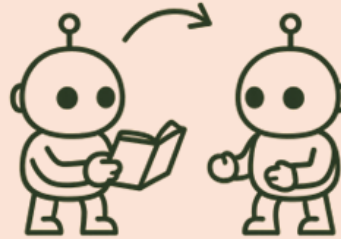
Operators Design

Counter



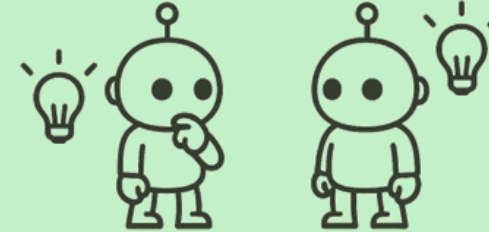
Generates a revision that exploits weaknesses in the opponent's current implementation

Learning

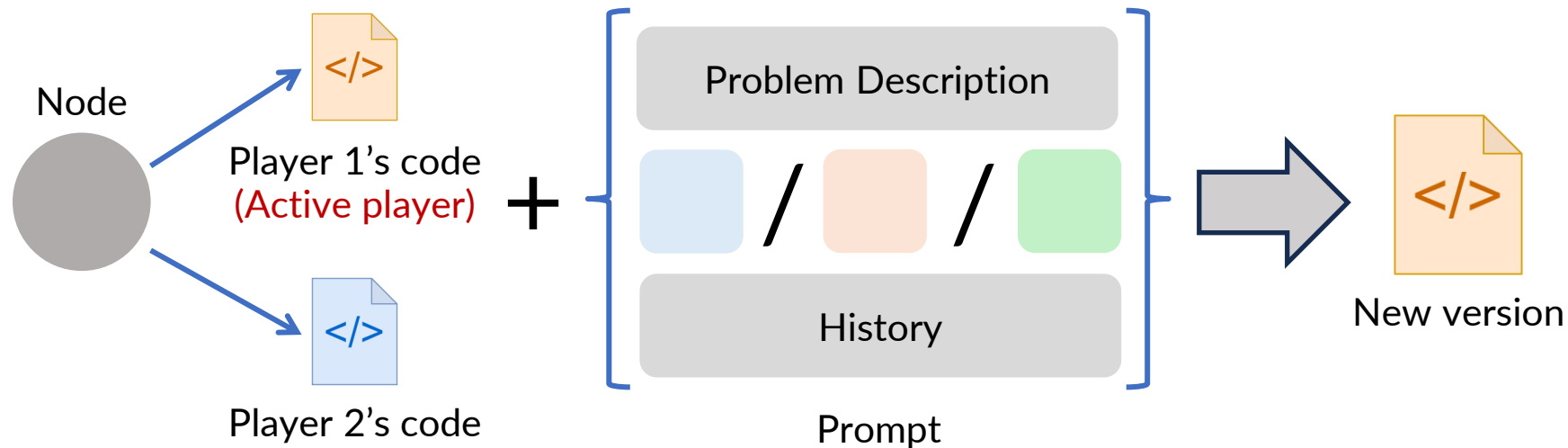


Integrates the opponent's successful ideas into a refined, strengthened implementation

Innovation



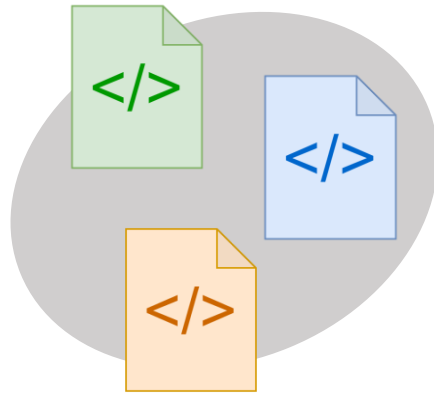
Introduces a new implementation that explores unexplored design directions



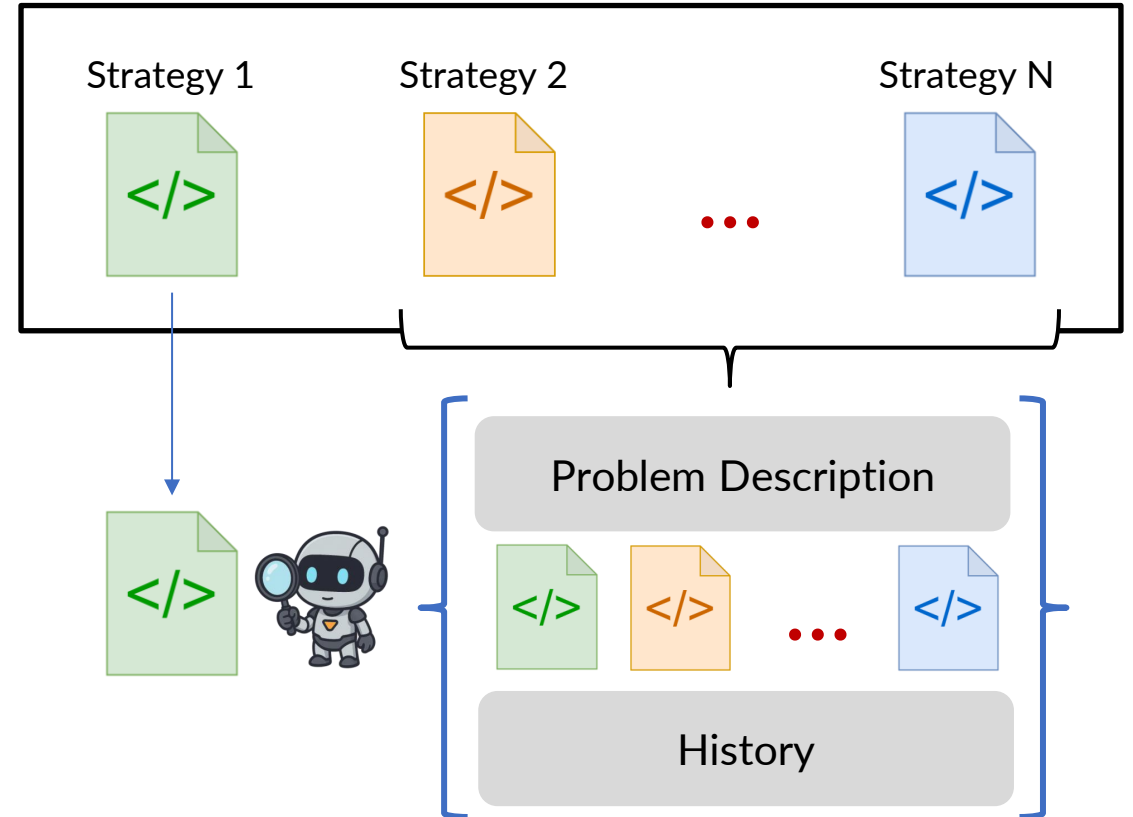
System-aware Refinement

- Refinement is performed with access to the complete solver configuration.
- Strategies are adjusted considering their interactions with other components.

Are these implementations well-aligned with each other?



Implementations
from round 1



4. Experimental Evaluation & Analysis

4.1. Algorithms and Benchmarks

4.2. Single-strategy Optimization Results

4.3. Multi-strategy Optimization Results

4.4. Convergence Behavior & Diversity

Algorithms/Solvers & Benchmarks

The compared baselines include EoH⁽⁴⁾, ReEvo⁽⁵⁾, and MCTS-AHD⁽⁷⁾.

Table 1. *Algorithms, problem domains, instance sizes, and training/testing splits used in the experiments*

GLS: Guided Local Search, **ACO:** Ant Colony Optimization, **DR:** Deconstruction and Repair

Algorithms	Problems	Instance size	#Train size	#Test size
GLS	TSP	200	5	64
ACO	TSP	50	5	64
	CVRP	50	5	64
	MKP	50	5	64
	OP	50	5	64
	BPP	50	5	64
DR	TSP	100	5	64
	CVRP	50	5	64
	BPP	100	5	64

(4) Liu et al. (2024), “Evolution of Heuristics: Towards Efficient Automatic Algorithm Design Using Large Language Model”, ICML 2024

(5) Ye et al. (2024), “ReEvo: Large Language Models as Hyper-Heuristics with Reflective Evolution, NeurIPS 2024

(7) Zheng et al. (2025), “Monte Carlo Tree Search for Comprehensive Exploration in LLM-Based Automatic Heuristic Design”, ICML 2025

Single-strategy Search

Table 2. Single-strategy search on GLS for TSP
The numbers indicate the optimality gap (%)

Methods	TSP50	TSP100	TSP200	TSP500
EoH	0.0000 $\pm$ 0.0000	0.0012 $\pm$ 0.0009	0.0639 $\pm$ 0.0097	0.5796 $\pm$ 0.0146
ReEvo	0.0000 $\pm$ 0.0000	0.0335 $\pm$ 0.0449	0.2081 $\pm$ 0.1927	0.7918 $\pm$ 0.3051
HSEvo	0.0108 $\pm$ 0.0138	0.3095 $\pm$ 0.2363	1.1254 $\pm$ 0.6424	2.4593 $\pm$ 0.7314
MCTS-AHD	0.0000 $\pm$ 0.0000	0.0024 $\pm$ 0.0007	0.0652 $\pm$ 0.0111	0.5611 $\pm$ 0.0216
MOTIF	0.0000 $\pm$ 0.0000	0.0007 $\pm$ 0.0006	0.0610 $\pm$ 0.0100	0.5577 $\pm$ 0.0255

Table 3. Comparison of single-strategy and multi-strategy search on DR across 3 COPs
The numbers indicate the optimality gap (%)

π_1 : initialization; π_2 : deconstruction, π_3 : repair

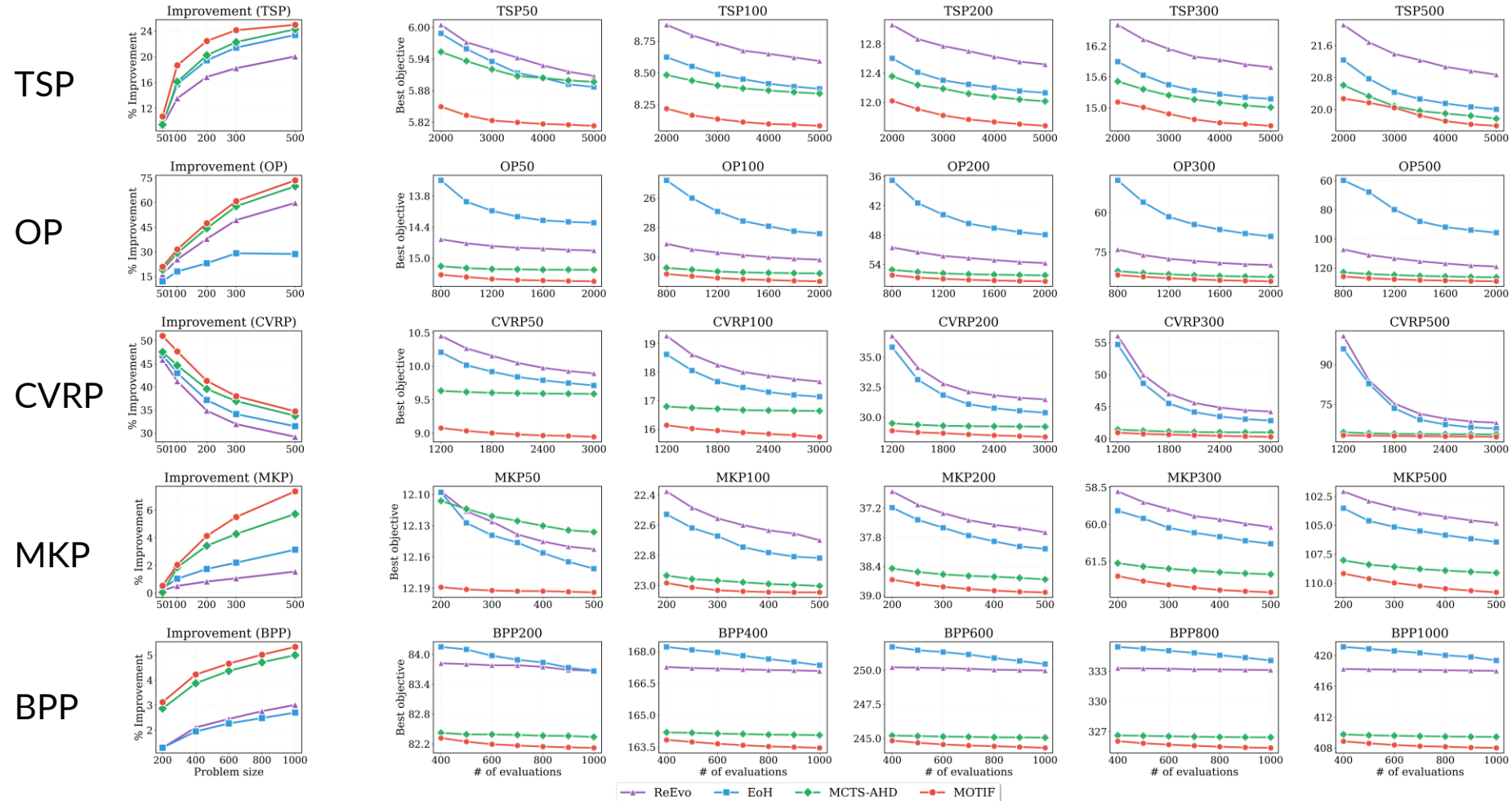
Strategies	TSP			CVRP			BPP		
	50	100	200	50	100	200	100	200	300
π_1	0.39 $\pm$ 0.17	0.81 $\pm$ 0.27	0.72 $\pm$ 0.45	1.47 $\pm$ 0.16	2.96 $\pm$ 0.25	2.30 $\pm$ 0.39	3.17 $\pm$ 0.00	4.82 $\pm$ 0.00	4.77 $\pm$ 0.00
π_2	3.13 $\pm$ 0.05	6.24 $\pm$ 0.21	8.18 $\pm$ 0.26	4.27 $\pm$ 0.24	7.04 $\pm$ 0.73	6.85 $\pm$ 0.75	23.19 $\pm$ 0.05	24.42 $\pm$ 0.02	25.00 $\pm$ 0.00
π_3	3.88 $\pm$ 0.04	7.91 $\pm$ 0.01	11.35 $\pm$ 0.03	6.34 $\pm$ 0.08	5.07 $\pm$ 0.02	4.28 $\pm$ 0.03	12.70 $\pm$ 7.51	12.84 $\pm$ 7.15	12.15 $\pm$ 7.49
(π_1, π_2)	2.58 $\pm$ 0.50	5.04 $\pm$ 0.78	5.84 $\pm$ 1.26	6.06 $\pm$ 1.64	7.94 $\pm$ 2.71	8.85 $\pm$ 2.64	23.15 $\pm$ 0.15	24.40 $\pm$ 0.06	24.98 $\pm$ 0.02
(π_2, π_3)	3.83 $\pm$ 0.16	7.97 $\pm$ 0.14	11.59 $\pm$ 0.07	10.64 $\pm$ 0.74	12.31 $\pm$ 0.57	11.71 $\pm$ 1.03	23.05 $\pm$ 0.11	24.32 $\pm$ 0.15	24.89 $\pm$ 0.15
(π_1, π_2, π_3)	3.88 $\pm$ 0.04	7.96 $\pm$ 0.03	11.65 $\pm$ 0.05	10.98 $\pm$ 1.64	12.84 $\pm$ 3.17	13.06 $\pm$ 3.67	23.94 $\pm$ 0.55	25.02 $\pm$ 0.41	25.41 $\pm$ 0.26

It shows that: multi-strategy > single-strategy

Multi-strategy Search

Multi-strategy search on ACO across 5 COPs

The numbers indicate the objective



Convergence & Diversity

(1) Stable convergence across players

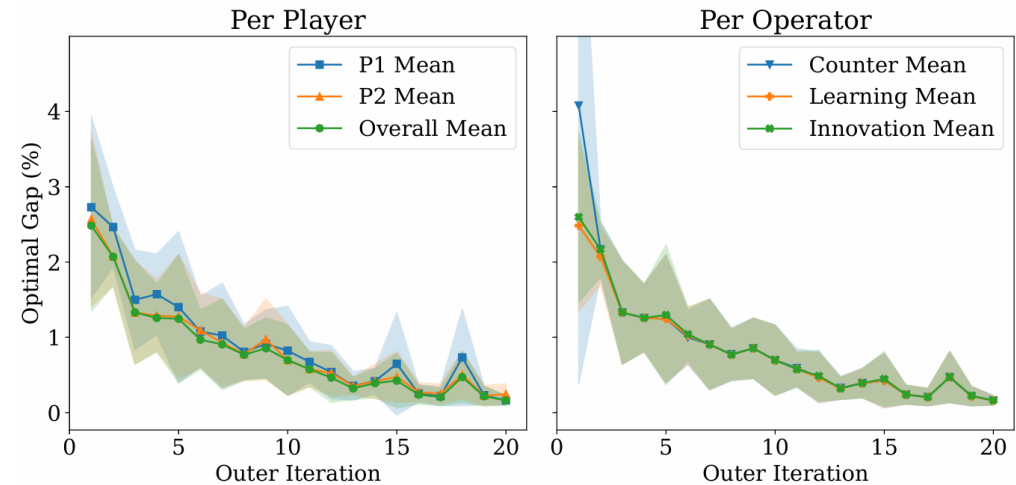
Both players converge at similar rates, indicating fair and stable self-play.

(2) All operators are effective, but serve distinct behavioral roles

- Counter: balance and stabilization
- Learning: refinement and exploitation
- Innovation: exploration and diversity

→ The combination of operators achieves a healthy trade-off between convergence and diversity.

Convergence behavior of the MOTIF framework during training in the ACO setting, averaged over five independent runs



Operator	Success rate ($\uparrow$)	Novelty score ($\uparrow$)	Silhouette score ($\uparrow$)
Counter	$93 \pm 2 \%$	0.0136 ± 0.0067	0.5121 ± 0.0208
Learning	$92 \pm 2 \%$	0.0118 ± 0.0054	0.5325 ± 0.0300
Innovation	$97 \pm 1 \%$	0.0175 ± 0.0110	0.4793 ± 0.0240

Thanks for your listening!